Project Report: AIML Weather Assistant

1. Introduction

The **AIML Weather Assistant** is a Python-based Artificial Intelligence application designed to predict weather conditions based on real-time user inputs of temperature and humidity.

Unlike traditional statistical machine learning models that require extensive datasets and trained weights (often saved as `.pkl` files), this project explores the implementation of a **Rule-Based Expert System**. By utilizing explicit, human-readable logical rules, the system mimics the decision-making process of a human meteorologist. Furthermore, the system features a dynamic data logging and visualization pipeline, allowing it to record historical data, calculate rolling averages, and generate trend graphs.

2. Objectives

- To develop a functioning Rule-Based Expert System using Python.
- To implement persistent data storage using CSV file handling.
- To create dynamic, real-time data visualizations using the `matplotlib` library.
- To apply modular programming principles by dividing the application into distinct, purposeful scripts (`app.py` , `model_logic.py` , and `analysis.py`).

3. System Architecture and Modules

The project is structured using a modular approach, separating the user interface, the AI logic, and the data visualization into distinct files.

3.1. `app.py` (Main Controller)

This is the central execution script. It serves as the bridge between the user, the AI logic, and the dataset. Its primary functions include:

- **Reading Historical Data:** It parses `dataset/history.csv` to calculate and display the historical average temperature and humidity.
- **Input Validation:** It utilizes `try/except` blocks to safely gather and validate user inputs.
- **Orchestration:** It passes the inputs to the logic module, receives the prediction, triggers the data saving process, and finally calls the visualization module.

3.2. `model_logic.py` (The AI Engine)

This module acts as the core "brain" of the expert system. Instead of relying on a pre-trained machine learning model, it contains a decision tree built from strict conditional statements. It evaluates the temperature (T) and humidity (H) to classify the weather into distinct categories (e.g., Snow, Cold, Rain, Sunny, Cloudy, or Normal).

3.3. `analysis.py` (Visualization Module)

This script utilizes the `matplotlib.pyplot` library to provide a visual representation of the stored data. It reads the updated CSV file and plots two separate line graphs side-by-side:

- Temperature trends over time (Day vs. Temp).
- Humidity trends over time (Day vs. Humidity).

3.4. `dataset/history.csv` (Data Storage)

A local CSV file that acts as the system's memory. Every time the program runs, the new day's temperature, humidity, and resulting weather prediction are appended as a new row, allowing the dataset to grow organically with user interaction.

4. Methodology: Rule-Based Logic vs. Statistical ML

A key design decision in this project was the use of a Rule-Based Expert System rather than a statistical model (like Linear Regression or a Random Forest).

In statistical ML, a model must be trained on thousands of rows of data to "learn" the mathematical correlations, which are then exported as a `.pkl` file. However, weather classification often follows strict meteorological thresholds. By using a rule-based approach, the AI's decision-making process is 100% transparent, requires zero training time, and guarantees predictable, accurate outputs based on the programmed expertise. Therefore, the logic script perfectly replaces the need for a `.pkl` file in this architecture.

5. Output and Results

When the application is executed, the following sequence occurs successfully:

1. **Averages Displayed:** The system accurately reads past entries and prints the historical averages to the terminal.
2. **Prediction Generated:** The system successfully processes new user inputs and prints the accurate weather prediction based on the logic constraints.
3. **Data Persistence:** The system successfully appends the new data to the CSV file without overwriting previous entries.
4. **Graphical Output:** A `matplotlib` window opens, successfully plotting the historical timeline of both temperature and humidity, providing an immediate visual context for the

data.

6. Conclusion and Future Scope

The AIML Weather Assistant successfully demonstrates how logical AI, data persistence, and data visualization can be integrated into a cohesive, modular Python application. The system is lightweight, highly accurate within its defined rules, and transparent in its operations.

Future enhancements could include:

- Expanding the rule base in `model_logic.py` to include wind speed and atmospheric pressure for more nuanced predictions.
- Transitioning from a terminal-based interface to a Graphical User Interface (GUI) using libraries like `Tkinter` or `PyQt`.
- Implementing a database system (like SQLite) to replace the CSV file for faster data